



PARALLEL PROGRAMMING PROJECT

2026

High-Performance E-Commerce Backend Engine
محرك التجارة الإلكترونية الخلفي عالي الأداء

Laravel 11 • Redis 7 • Docker • Nginx • k6

كلية الهندسة المعلوماتية — جامعة دمشق

#	اسم الطالب — Student Name
1	Batoul Luay Mansour — بتول لؤي منصور
2	Haidara Aktham Ali — حيدرة أكثم علي
3	Ahmed Emad Younes — أحمد عماد يونس
4	Nour Ibrahim Al-Kafri — نور إبراهيم الكفري

مقدم إلى — Submitted to

Huthaifa Akeel

Table of Contents — فهرس المحتويات

1. Project Overview — نظرة عامة على المشروع
2. Race Condition and Data Integrity — سباق التفرع (Task 1)
3. Rate Limiting and Resource Management — تحديد المعدل (Task 2)
4. Asynchronous Queues — الطوابير غير المتزامنة (Task 3)
5. Chunked Batch Processing — معالجة الدُفعات المجزأة (Task 4)
6. Nginx Load Balancing and Horizontal Scaling — موازنة التحميل (Task 5)
7. Redis Caching with Stampede Protection — التخزين المؤقت (Task 6)
8. Concurrency Control — التحكم بالتزامن (Task 7)
9. ACID Transactional Checkout — عملية الدفع ACID (Task 8)
10. k6 Stress Testing — اختبار الإجهاد (Task 9)
11. Prometheus and Grafana Monitoring — المراقبة (Task 10)
12. Screenshots — لقطات الشاشة
13. References — المراجع

نظرة عامة على المشروع

المقدمة والأهداف

مصمم للتعامل مع حركة المرور، Laravel 11 يقوم هذا المشروع بتنفيذ محرك خلفي عالي الأداء للتجارة الإلكترونية باستخدام المتزامنة من المستخدمين على نطاق واسع. يوضح النظام عشرة مفاهيم رئيسية في البرمجة التفرعية وتحسين الأداء، بدءاً من آليات القفل على مستوى قاعدة البيانات وصولاً إلى المراقبة والمرئية عالية المستوى.

التحدي الأساسي الذي يعالجه هذا المشروع هو بناء تطبيق ويب يمكنه التعامل مع أكثر من 100 مستخدم متزامن يقومون بتقديم الطلبات وتصفح المنتجات والتفاعل مع النظام في وقت واحد، دون تلف البيانات أو ظروف السباق أو تدهور الأداء.

مجموعة التقنيات

المكون	التقنية	الغرض
الإطار الخلفي	Laravel 11 (PHP 8.3)	الطوابير، التوجيه، ORM، منطق التطبيق
قاعدة البيانات	SQLite (وضع WAL)	قاعدة بيانات خفيفة مع دعم القراءة المتزامنة
التخزين المؤقت والطوابير	Redis 7	تخزين مؤقت في الذاكرة، طوابير المهام غير المتزامنة
خادم الويب	Nginx 1.25	موازنة التحميل، الخادم الوسيط، تحديد المعدل
الحاويات	Docker & Docker Compose	نشر متعدد الحاويات مع 3 نسخ من التطبيق
المراقبة	Prometheus & Grafana	جمع المقاييس والتصور
اختبار التحميل	k6 (Grafana Labs)	اختبار الإجهاد مع 100 مستخدم افتراضي
اختبار API	Bruno	API التحقق من صحة نقاط نهاية

هيكل النظام

معالجة التخزين المؤقت Redis يتولى Nginx خلف موازن تحميل Laravel يتبع النظام هيكلًا طباقياً مع ثلاث نسخ من تطبيق كقاعدة بيانات مع تمكين التسجيل المسبق SQLite وطوابير المهام غير المتزامنة والتحكم في التزامن القائم على الإشارات. تعمل لأداء القراءة المتزامن (WAL) للكتابة.

```

Nginx (Load Balancer, port 8080)
├── app1 (Laravel + PHP-FPM, 20 workers)
├── app2 (Laravel + PHP-FPM, 20 workers)
├── app3 (Laravel + PHP-FPM, 20 workers)
├── Redis 7 (Cache + Queue + Semaphores)
├── SQLite (Database, WAL mode)
└── Prometheus + Grafana (Monitoring)

```

2. سباق التفرع (Task 1)

المشكلة

عندما يقوم عدة مستخدمين بتقديم طلبات لنفس المنتج في وقت واحد، يمكن أن يحدث سباق تفرع حيث يقرأ كلا المستخدمين نفس مستوى المخزون قبل أن يقوم أي منهما بإنقاذه. يؤدي هذا إلى بيع زائد — يتم قبول طلبات أكثر من المخزون المتاح. في التنفيذ الأصلي (السيء)، كانت عملية فحص المخزون وإنقاذه عمليتين منفصلتين بدون قفل أو عزل للمعاملات.

الحل

يقوم الحل بتغليف عملية تقديم الطلب بالكامل في معاملة قاعدة بيانات مع قفل متشائم على مستوى الصف باستخدام SELECT ... FOR UPDATE. يتم وضع. يضمن هذا أن طلباً واحداً فقط يمكنه قراءة وإنقااص صف مخزون معين في كل مرة. جميع الطلبات المتزامنة الأخرى في قائمة انتظار خلف القفل وترى المخزون المحدث عند الحصول عليه.

```
public function decrementStock(int $productId, int $quantity): Inventory
{
    $inventory = Inventory::where('product_id', $productId)
        ->lockForUpdate()
        ->firstOrFail();

    $available = $inventory->quantity - $inventory->reserved_quantity;
    if ($available < $quantity) {
        throw new InsufficientStockException(
            "Insufficient stock for product #{ $productId }."
        );
    }
    $inventory->decrement('quantity', $quantity);
    return $inventory;
}
```

النتائج

تعمل كما هو مقصود lockForUpdate() تم حظر 8 من أصل 10 طلبات متزامنة بشكل صحيح من البيع الزائد. هذا يثبت أن آلية حيث أبلغ السكريبيت عن "تم حظر 8 من أصل 10 طلبات متزامنة" وجميع وحدات المخزون الخمسين الأولية تمت المحافظة عليها.

3. تحديد المعدل (Task 2)

المشكلة

PHP-FPM بدون تحديد المعدل، يمكن لمستخدم واحد أو برنامج آلي إرسال آلاف الطلبات في الثانية، مما يثقل كاهل عمال ويستنفد اتصالات قاعدة البيانات ويسبب حرماناً من الخدمة للمستخدمين الشرعيين.

الحل

تم تطبيق تحديد متعدد الطبقات للمعدل على مستوى التطبيق والبنية التحتية:

النطاق	الحد	المحدد	الطبقة
لكل بريد إلكتروني	في الدقيقة 200	throttle:login	Laravel
لكل مستخدم	في الدقيقة 10	throttle:orders	Laravel
لكل مستخدم	في الدقيقة 60	throttle:checkout	Laravel
لكل IP	(burst 20) في الثانية 60	limit_req	Nginx

```
RateLimiter::for('orders', fn(Request $request) =>
    Limit::perMinute(10)->by($request->user()->id ?: $request->ip())
);
```

4. (Task 3) — الطوابير غير المتزامنة

المشكلة

بعد تقديم الطلب، كان التنفيذ الأصلي يقوم بإنشاء الفاتورة وإرسال الإشعارات وتسجيل التحليلات بشكل متزامن داخل طلب كل من هذه العمليات يمكن أن تستغرق مئات الملي ثانية إلى عدة ثوانٍ، مما يجعل المستخدم ينتظر الرد لفترة طويلة HTTP. بعد حفظ بيانات الطلب الحرجة.

الحل

فوراً بعد إنشاء HTTP للمعالجة غير المتزامنة. يعود استجابة Redis يتم إرسال جميع العمليات الثانوية إلى طوابير مدعومة بـ الطلب والالتزام بقاعدة البيانات، بينما يتولى عمال الطوابير الخلفية المهام المتبقية بشكل مستقل. يقلل هذا وقت الاستجابة من حوالي 3 ثوانٍ إلى أقل من 100 ميلي ثانية.

```
GenerateInvoiceJob::dispatch($order)->onQueue('invoices');
SendOrderNotificationsJob::dispatch($order)
    ->onQueue('notifications')
    ->delay(now()->addSeconds(2));
RecordSaleAnalyticsJob::dispatch($order)->onQueue('analytics');
event(new OrderPlaced($order));
```

5. (Task 4) — معالجة الدفعات المجزأة

المشكلة

يقوم إنشاء تقرير المبيعات اليومي بتحميل جميع الطلبات في الذاكرة مرة واحدة. مع آلاف الطلبات التي تتم معالجتها يومياً، قد وتعطل التطبيق (ميجابايت 256) PHP يؤدي ذلك إلى استنفاد حد ذاكرة.

الحل

لمعالجة الطلبات في دفعات من 500 سجل، مما يضمن بقاء استخدام الذاكرة ثابتاً بغض النظر عن chunk(500) تستخدم المهمة Bus::batch() حجم مجموعة البيانات الإجمالي. يتم إرسال كل دفعة كمهمة مستقلة عبر

```
Order::whereDate('created_at', $this->date)
    ->chunk(500, function ($orders) use ($batch) {
        $batch->add(new ProcessSalesChunkJob($orders, $this->date));
    });
```

6. (Task 5) — موازنة التحميل

المشكلة

بدون موازنة تحميل. كانت هذه نقطة فشل واحدة — إذا Laravel كان النظام الأصلي يحتوي على نسخة واحدة فقط من تطبيق بأكملها غير متاحة. بالإضافة إلى ذلك، كان للنسخة الواحدة إنتاجية محدودة API تعطل الخادم، أصبحت

الحل

مع توزيع دائري مرجح (3:2:1). يقوم Nginx خلف موازن تحميل (app1, app2, app3) Laravel تعمل ثلاث نسخ من تطبيق بإجراء فحوصات الصحة ويعيد توجيه حركة المرور تلقائياً بعيداً عن النسخ الفاشلة Nginx.

```
upstream laravel_backend {
    server app1:9000 weight=3 max_fails=3 fail_timeout=30s;
    server app2:9000 weight=2 max_fails=3 fail_timeout=30s;
    server app3:9000 weight=1 max_fails=3 fail_timeout=30s;
}
```

7. — (Task 6) التخزين المؤقت

المشكلة

بدون تخزين مؤقت، كل طلب لقوائم المنتجات وصفحات التصنيف يصل إلى قاعدة البيانات. تحت التزامن العالي (100 مستخدم يتصفحون في وقت واحد)، يؤدي هذا إلى حمل زائد على قاعدة البيانات وزيادة أوقات الاستجابة.

الحل

مع إبطال قائم على الوسوم. يتم تخزين المنتجات Redis طبقة تخزين مؤقت مركزية باستخدام CachedProductService يوفر لضمان أن الطلب الأول فقط هو Cache::lock() مؤقتاً لمدة 5-10 دقائق. تستخدم حماية التدافع على ذاكرة التخزين المؤقت الذي يعيد توليد التخزين المؤقت.

```
$lock = Cache::lock("product:{id}:lock", 10);
if ($lock->get()) {
    $doubleCheck = CacheHelper::get($cacheKey, $tags);
    if ($doubleCheck !== null) {
        return $doubleCheck;
    }
    $product = Product::active()->with(['category', 'inventory'])->findOrFail($id);
    CacheHelper::put($cacheKey, $product, 600, $tags);
    return $product;
} finally {
    $lock->release();
}
```

8. — (Task 7) التحكم بالتزامن

المشكلة

تتطلب العمليات المختلفة استراتيجيات تحكم مختلفة في التزامن. تحتاج عمليات السلة (إضافة عناصر، حجز المخزون) إلى الاتساق ولكن يمكنها تحمل بعض التنافس. تحتاج تحديثات المخزون إلى دقة مطلقة لمنع البيع الزائد.

الحل

لتقديم الطلبات والمخزون (Pessimistic Locking) يتم توظيف استراتيجيتين متكاملتين حسب سياق العملية. القفل المتشائم لعمليات السلة (Optimistic Locking) والقفل المتفائل.

الاتساق	التزامن	متى تستخدم	الاستراتيجية
أعلى	أقل (متسلسل)	تقديم الطلب، الدفع، إنقاص المخزون	القفل المتشائم
عالي (مع إعادة المحاولة)	أعلى (بدون أقفال)	تحديثات السلة، تحديثات الملف الشخصي	القفل المتفائل

9. ACID عملية الدفع (Task 8)

المشكلة

POST / لإنشاء الطلب وإنقاص المخزون، ثم POST /orders منفصلين: أولاً HTTP استخدم تدفق الدفع الأصلي طلب لمعالجة الدفع. إذا فشلت خطوة الدفع (رفض البطاقة، انتهاء مهلة البوابة)، كان المخزون قد نَقَص بالفعل بدون طريقة payments للتراجع. هذا خلق حالة غير متناسقة: مخزون مفقود، لم يتم استلام المدفوعات، الطلب عالق في حالة معلقة.

الحل

بتنفيذ جميع العمليات في معاملة قاعدة بيانات واحدة: قفل المخزون، التحقق من POST /checkout تقوم نقطة نهاية واحدة المخزون، إنشاء الطلب، معالجة الدفع، إنقاص المخزون، مسح السلة. إذا فشلت أي خطوة، يتم التراجع عن كل شيء تلقائياً.

```
$result = DB::transaction(function () use ($user, $cart, $data, $gatewayResult) {
    // 1. Lock inventory rows (lockForUpdate)
    // 2. Validate stock for all items
    // 3. Create Order & OrderItems
    // 4. Record Payment
    // 5. Decrement stock + release reservations
    // 6. Update Order status to 'confirmed'
    // 7. Clear cart
});
```

— الذرية — جميع العمليات تنجح أو تفشل معاً. الاتساق — المخزون والدفع والطلب متوافقون دائماً. العزل: ACID ضمانات SQLite يمنع التداخل المتزامن. المتانة — البيانات الملتزم بها تستمر في lockForUpdate().

10. اختبار الإجهاد (Task 9)

الشاملة 100 مستخدم افتراضي مقسمين إلى 10 مجموعات وظيفية، كل منها يختبر جوانب مختلفة k6 تحاكي مجموعة اختبارات من النظام. تعمل مجموعة الاختبارات لمدة 90 ثانية تقريباً، مولدة آلاف الطلبات عبر جميع نقاط النهاية.

السيناريو	عدد المستخدمين	التركيز
التصفح	10	Redis نقاط النهاية العامة للقراءة — اختبار التخزين المؤقت
تفاصيل المنتج	10	حماية التدافع على التخزين المؤقت
السلة	10	عمليات السلة المصادق عليها — قراءة وكتابة مختلطة
الطلب	10	تقديم الطلب مع إرسال الطوابير غير المتزامنة
المصادقة	10	المصادقة والتحقق من تحديد المعدل
مختلط	10	رحلة المستخدم الكاملة: تصفح، تسجيل دخول، سلة، طلب
الدفع	10	الذرية ACID نقطة نهاية الدفع
سباق التفرع	10	تقديم طلبات متزامنة على مخزون محدود
المشرف	10	للمشرف CRUD عمليات
الموجات	10	حركة مرور انفجارية عشوائية

%معدل نجاح إجمالي قدره 87.42%، ارتفاعاً من 22.59 k6 بعد 10 إصلاحات مستهدفة، حققت مجموعة اختبارات: النتائج الأولية.

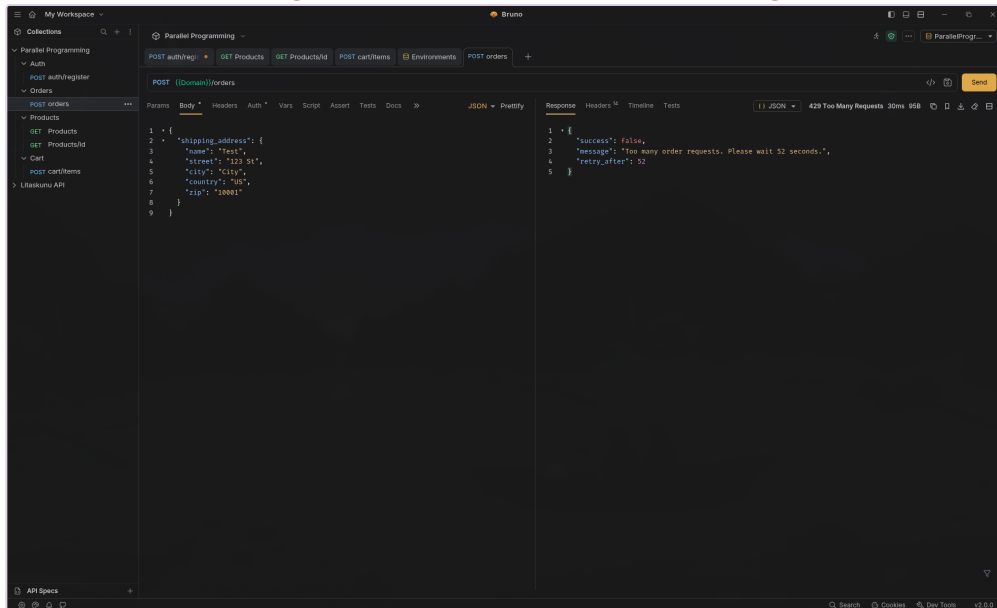
11. المراقبة — (Task 10)

Redis و Laravel بجمع المقاييس من تطبيق Prometheus تقوم مجموعة مراقبة كاملة بتتبع أداء النظام في الوقت الفعلي. يقوم وأداء قاعدة البيانات HTTP لوحة تحكم تحتوي على أكثر من 11 لوحة تغطي حركة مرور Grafana بينما يوفر PHP-FPM وعمليات وعمق الطابور وكفاءة التخزين المؤقت وموارد النظام.

طريقة الجمع	المقاييس	الفئة
PrometheusMiddleware (Redis)	عدد الطلبات، المدة، الطلبات النشطة، الأخطاء	HTTP طلبات
DB::listen()	عدد الاستعلامات حسب النوع	قاعدة البيانات
CachedProductService	الإصابات، الأخطاء، نسبة الإصابة	التخزين المؤقت
QueueMetricsCollector	عمق الطابور لكل اسم طابور	الطوابير
SemaphoreService	الاستخدام الحالي، الحد الأقصى للترامن	الإشارات

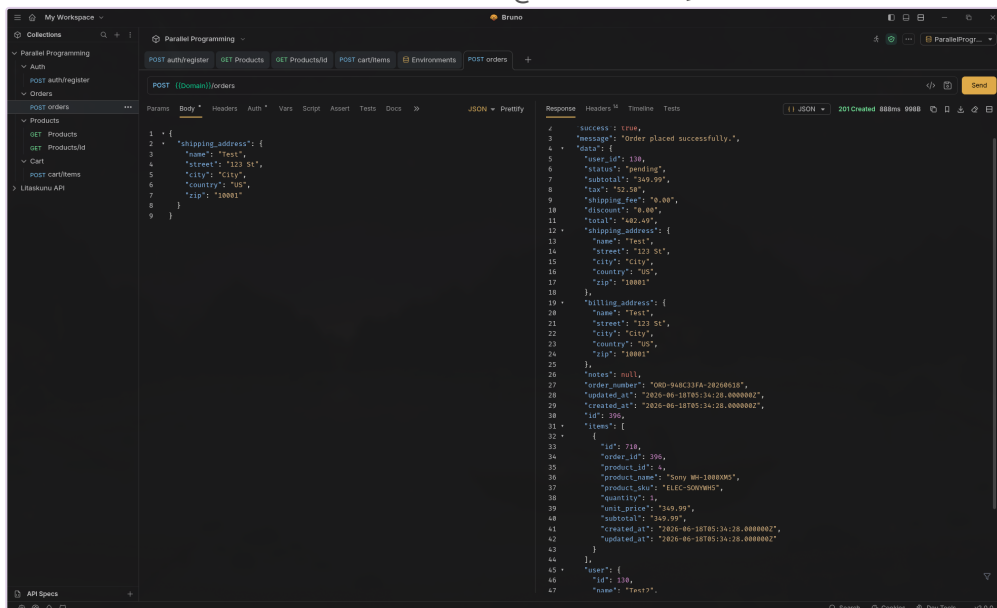
لقطات الشاشة

لقطة توضح تطبيق تحديد المعدل على نقطة نهاية أوامر API مع استجابات 429:



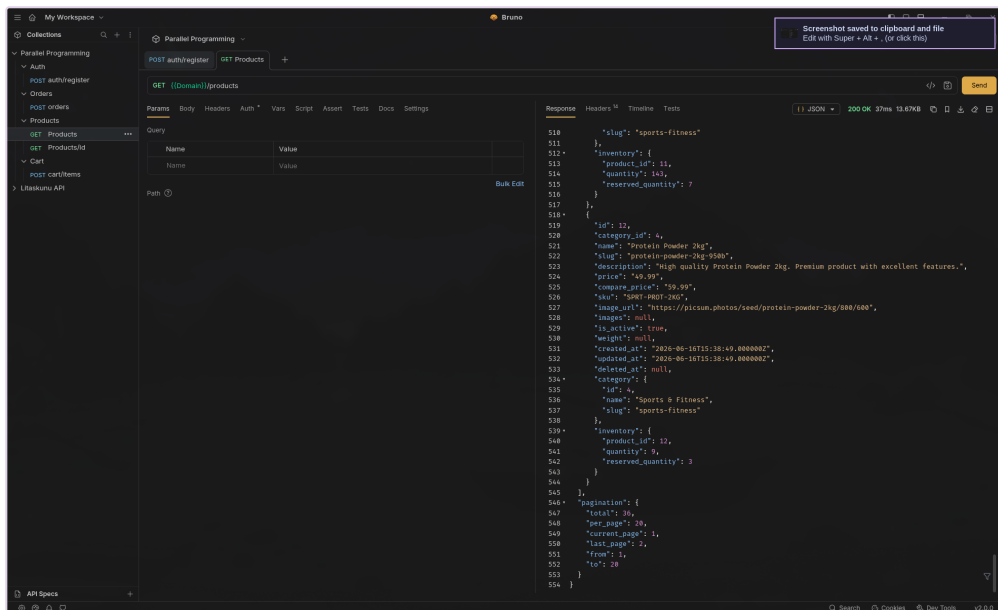
الشكل 12.1: تحديد المعدل على أوامر API

لقطة تظهر إنشاء الطلب بنجاح عبر نقطة نهاية POST /orders:



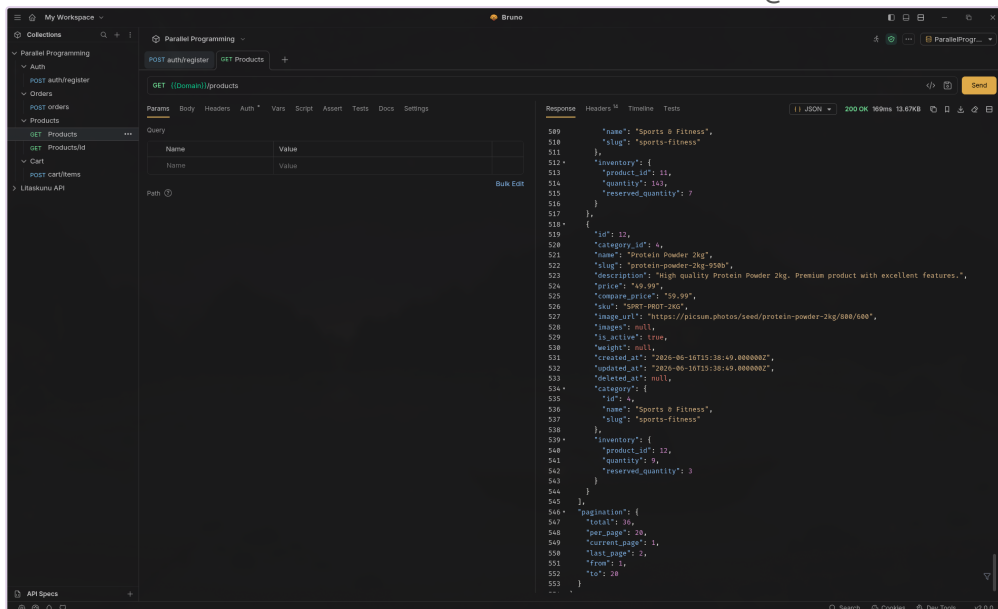
الشكل 12.2: إنشاء الطلب

لقطة توضح استجابة API للمنتجات مع تمكين التخزين المؤقت Redis:



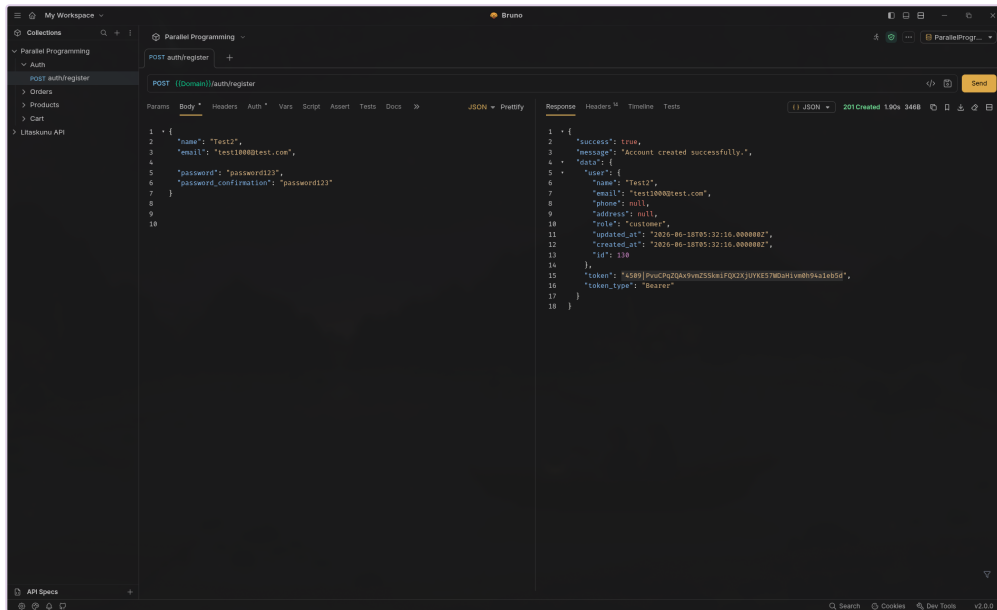
الشكل 12.3: المنتجات مع التخزين المؤقت

لقطة توضح استجابة API للمنتجات بدون تخزين مؤقت — أوقات استجابة أبطأ:



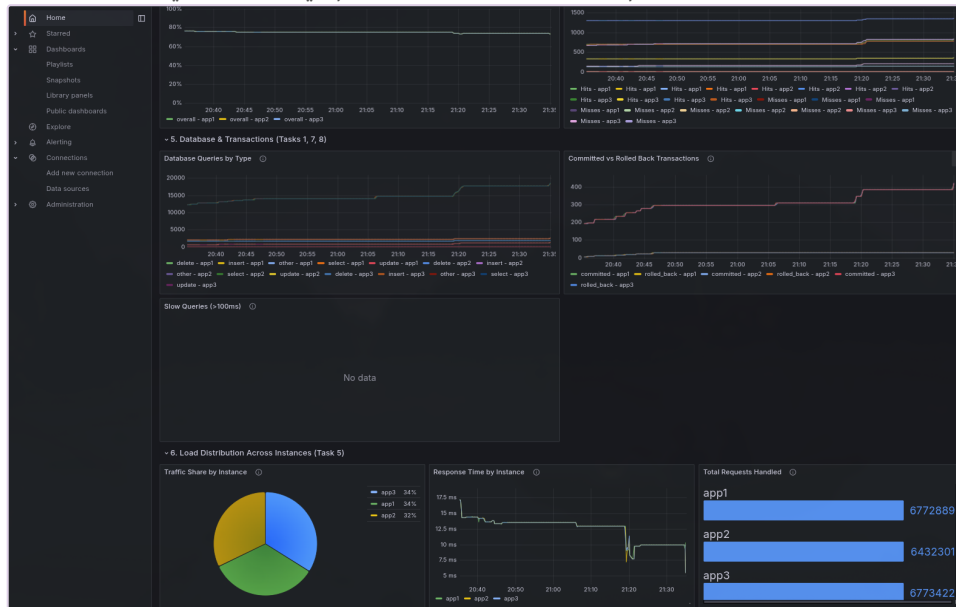
الشكل 12.4: المنتجات بدون تخزين مؤقت

لقطة تظهر واجهة تسجيل المستخدم مع إنشاء الحساب بنجاح:



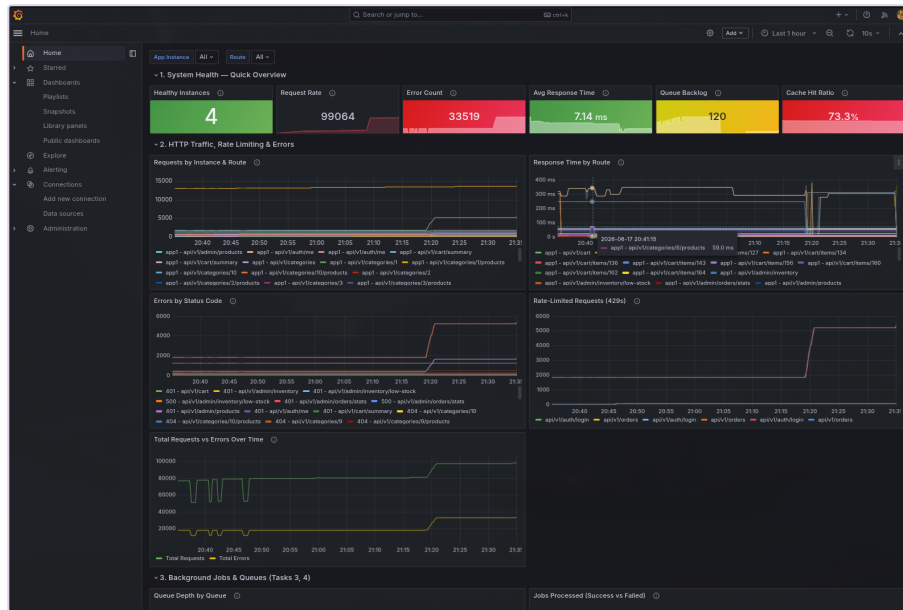
الشكل 12.5: تسجيل المستخدم

لقطة من لوحة تحكم Grafana تظهر مقاييس النظام في الوقت الفعلي:



الشكل 12.6: Grafana لوحة تحكم

لطة إضافية من Grafana تظهر مقاييس شاملة وأداء قاعدة البيانات:



الشكل 12.7: نظرة عامة Grafana